

Trabalho Prático 1 – Alinhamento Estratégico de Carteiras

Disciplina: Projeto e Análise de Algoritmos | **Docente:** Prof. Douglas Castilho

Instituição: IFSULDEMINAS – Campus Poços de Caldas | **Curso:** Engenharia de Computação (6º Período)

1. Descrição do Problema

No mercado financeiro, corretoras e casas de análise (*research houses*) publicam trimestralmente relatórios de alocação de ativos contendo listas ordenadas com tickers de ações da B3 (como PETR4, VALE3, ITUB4) recomendadas para compra, dispostas em ordem estrita de preferência e prioridade.

Uma gestora de investimentos busca instituir a estratégia do "**Núcleo de Consenso Ordenado**" entre duas corretoras distintas (Corretora A e Corretora B). O objetivo é determinar a **maior sequência de ações em comum que preserva rigorosamente a ordem relativa de recomendação em ambas as casas**. Por exemplo, se a Corretora A recomenda comprar PETR4 antes de ITUB4 e a Corretora B também recomenda PETR4 antes de ITUB4 (mesmo havendo outros ativos intercalados), a sequência [PETR4, ITUB4] é válida.

Entrada e Saída: A primeira linha da entrada contém dois inteiros M e N ($1 \leq M, N \leq 1000$), indicando respectivamente o número de ações da Corretora A e da Corretora B. As linhas seguintes contêm os códigos de ativos das corretoras A e B, respectivamente. A leitura pode ser feita via arquivo de texto ou da entrada padrão (*stdin*). Na saída, a primeira linha informa o inteiro K (quantidade máxima de ativos em consenso) e a segunda linha exibe os K tickers ordenados, separados por espaço. Caso não haja ativos em comum ($K = 0$), imprime-se $\emptyset$ na primeira linha e uma linha em branco na segunda.

2. Modelagem da Solução e Referencial Teórico (CLRS)

O problema formulado mapeia-se de maneira direta e canônica no problema clássico da **Maior Subsequência Comum (Longest Common Subsequence – LCS)**, amplamente estudado no livro *Introduction to Algorithms* (CLRS – Cormen, Leiserson, Rivest e Stein).

Por que LCS? Uma *subsequência* preserva a ordem relativa dos elementos sem a exigência de que sejam contíguos (ao contrário de uma substring). Como as corretoras podem intercalar ações distintas entre recomendações coincidentes, a carteira de consenso desejada é exatamente a maior subsequência comum entre as duas sequências de tickers.

Por que Programação Dinâmica? Uma busca por força bruta (testando todas as $2^{\min(M, N)}$ subsequências) seria inviável. O problema satisfaz as duas propriedades essenciais da Programação Dinâmica:

- **Subestrutura Ótima:** Uma LCS ótima das sequências completas contém em seu interior a LCS ótima dos prefixos menores. Se os últimos ativos coincidem, ele integra a LCS final somando-se à solução dos prefixos restantes. Se diferem, a solução ótima é a melhor opção entre omitir o último ativo de A ou o último ativo de B.
- **Subproblemas Sobrepostos:** As escolhas recursivas geram chamadas repetidas sobre os mesmos prefixos de listas. A programação dinâmica resolve cada subproblema uma única vez por tabulação (*bottom-up*), armazenando os valores calculados em uma tabela bidimensional para acesso em tempo $O(1)$.

3. Explicação Detalhada do Algoritmo

Significado de `dp[i][j]`

Define-se uma matriz `dp` de dimensões $(M + 1) \times (N + 1)$, onde cada elemento `dp[i][j]` representa o **comprimento da maior subsequência comum** considerando os primeiros i ativos da Corretora A e os primeiros j ativos da Corretora B.

Relação de Recorrência

Casos Base:

`dp[i][0] = 0`, para todo $0 \leq i \leq M$

`dp[0][j] = 0`, para todo $0 \leq j \leq N$

Passo Indutivo (para $1 \leq i \leq M$ e $1 \leq j \leq N$):

- Se `A[i - 1].equals(B[j - 1])`: `dp[i][j] = dp[i - 1][j - 1] + 1`
- Se `A[i - 1]` diferente de `B[j - 1]`: `dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])`

Construção da Tabela e Reconstrução da Resposta

A tabela é preenchida iterativamente por dois laços `for` simples (i de 1 até M , e j de 1 até N). Ao término, o tamanho ótimo K é encontrado em `dp[M][N]`.

A reconstrução da sequência de ações é realizada por *backtracking* partindo de (M, N) até a borda da matriz ($i = 0$ ou $j = 0$):

- Se `A[i - 1].equals(B[j - 1])`, o ativo compõe a solução: ele é guardado e recua-se pela diagonal (`i--` e `j--`).
- Caso contrário, segue-se o caminho do valor máximo: se `dp[i - 1][j] > dp[i][j - 1]`, recua-se para cima (`i--`); senão, recua-se para a esquerda (`j--`).

Como o *backtracking* percorre a solução do final para o início, aloca-se um vetor de tamanho K e preenche-se de trás para frente (índice $K - 1$ até 0), restaurando a ordem original de forma direta e sem inversões adicionais.

4. Análise de Complexidade Assintótica

4.1. Complexidade de Tempo

- Leitura da Entrada:** Leitura de $M + N$ tokens via `Scanner` em tempo linear $\Theta(M + N)$.
- Construção da Tabela DP:** A matriz tem $(M + 1) \times (N + 1)$ células. Cada célula executa operações de comparação e indexação em tempo $O(1)$. Custo: $\Theta(M \times N)$.
- Reconstrução da Resposta:** O laço de retorno executa no máximo $M + N$ passos, cada um com custo $O(1)$. Custo: $O(M + N)$.
- Impressão da Saída:** Impressão de K ativos, onde $K \leq \min(M, N)$. Custo: $O(K)$.

Complexidade de Tempo Total: $O(M \times N)$.

Para os limites do problema ($M, N \leq 1000$), $M \times N \leq 10^6$ operações, executadas em menos de 20 milissegundos na JVM.

4.2. Complexidade de Espaço

- Vetores de Entrada:** Armazenam os $M + N$ tickers, ocupando $O(M + N)$.
- Tabela de DP:** Matriz de inteiros de tamanho $(M + 1) \times (N + 1)$. Para $M = 1000$, $N = 1000$, são aproximadamente 1 milhão de inteiros primitivos (~4 MB de memória), valor mínimo e perfeitamente gerenciável pela JVM. Custo: $O(M \times N)$.
- Vetor de Resposta:** Ocupa $O(K) \leq O(\min(M, N))$.

Complexidade de Espaço Total: $O(M \times N)$.

Justificativa: A alocação da matriz completa é a abordagem clássica adotada no CLRS, mantendo a

simplicidade e a legibilidade do algoritmo, permitindo uma reconstrução de resposta direta sem exigir algoritmos adicionais complexos.

5. Casos de Teste Executados

Caso de Teste	Dimensões	Resultado (K)	Sequência Obtida	Status
Exemplo 1 (PDF)	M=5, N=6	3	PETR4 ITUB4 ABEV3	Sucesso
Exemplo 2 (PDF - Disjuntos)	M=4, N=4	0	(linha em branco)	Sucesso
Exemplo 3 (PDF - Empate)	M=6, N=6	3	VALE3 BBDC4 WEGE3	Sucesso
Teste 1: Sem ativos em comum	M=3, N=3	0	(linha em branco)	Sucesso
Teste 2: Apenas um comum	M=4, N=3	1	ITUB4	Sucesso
Teste 3: Listas iguais	M=5, N=5	5	PETR4 VALE3 ITUB4 BBDC4 WEGE3	Sucesso
Teste 4: Tamanhos diferentes	M=2, N=7	2	PETR4 WEGE3	Sucesso
Teste 5: Vários intercalados	M=8, N=8	7	VALE3 ITUB4 BBDC4 ABEV3 WEGE3 RENT3 MGLU3	Sucesso
Teste 6: Ordem invertida	M=4, N=4	1	BBDC4	Sucesso
Teste 7: Limite Superior	M=1000, N=1000	500	500 tickers intercalados (Tempo < 200 ms)	Sucesso

6. Descrição do Uso de Inteligência Artificial

Em conformidade com as exigências do Trabalho Prático 1 e com os princípios de integridade acadêmica, relata-se o uso de Inteligência Artificial como assistente de desenvolvimento:

- **Ferramenta:** Agente de IA Antigravity (Google DeepMind).
- **Modelagem Teórica:** Auxílio na correlação dos conceitos do problema com o referencial do CLRS (subestrutura ótima e subproblemas sobrepostos em LCS).
- **Implementação:** Auxílio na escrita de código Java limpo, simples e estruturado, utilizando estruturas básicas (arrays, tipos primitivos, laços for/while) apropriadas para a graduação.
- **Tratamento de Robustez:** Identificação e tratamento do caractere BOM (Byte Order Mark) na leitura de arquivos UTF-8 no Windows.
- **Verificação e Testes:** Criação de casos de teste de cobertura e validação dos limites do enunciado.